Aarhus University

Department of Electrical and Computer Engineering

P1: Leader Election Project Report

Distributed Systems

Team 8 | Version v1.0

Authors

Name	Student No.	Email
Alexander Anton Rosenkilde	202409649	202409649@post.au.dk
Nicolas Berwald Sørensen	202408628	202408628@post.au.dk
Simon Tautra Grønseth	202407442	202407442@post.au.dk
Sophie Richardy Christensen	202406954	202406954@post.au.dk

Document Details

Hand-in Date 30-10-2026

Contents

1 Preface **1**
2 Introduction **1**
3 Methods and Materials **2**
3.1 The original Bully Algorithm . . . 2
3.2 The improved Bully Algorithm . . . 3
4 Design and Implementation **3**
4.1 Design . . . 3
4.1.1 Architecture . . . 3
4.1.2 Communication between nodes . . . 4
4.1.3 Node behavior . . . 4
4.2 Implementation . . . 5
4.2.1 Code structure . . . 5
4.2.2 Relevant Code snippet . . . 5
5 Experiments **6**
5.1 Experiment setup . . . 6
5.2 Scenarios . . . 7
5.3 Metrics . . . 7
6 Results **8**
6.1 Election outcome . . . 8
6.2 Message complexity . . . 8
7 Discussion **9**
7.1 Time- and message complexity . . . 9
7.2 Limitations . . . 9
8 Conclusion **9**
9 Perspectives **10**

1 Preface

In this report, we will focus on the first of two projects. The projects are 'P1: Leader election', and 'P1: Kubernetes'. This Report for the project 'Leader Election' will explain the leader election algorithm 'Bully algorithm' and an improved version of the algorithm, that we will define. The report will not focus on the Docker implementation and use of Kubernetes. It will focus on the bully algorithms and the implementation in Python. The report for project 'P1: Kubernetes' will provide the implementation of 'Bully' using Docker and Kubernetes.

AI was used to generate code for graphs in results, and for support with the code syntax in Python, as well as support for creating test files. The overall logic and decisions regarding methods were made by the authors of this report, as well as the implementation of both bully algorithms.

2 Introduction

Distributed systems often need a process to act as a coordinator that hands out tasks, holds locks and decides how updates should happen. In distributed systems in general, a process may crash at any given point. This also includes a coordinator. When that happens the remaining processes must be able to agree on a new coordinator on their own. Algorithms that solve this are called *leader election algorithms* [6]. Those algorithms come in many forms, but one

of the most popular is the Bully Algorithm, which was published by Garcia-Molina [1]. The algorithm will successfully elect the process with the highest ID but it comes with a cost of message complexity. A classic Bully Algorithm will have $O(n^2)$ message complexity in the worst case. This is something that can be improved.

This report covers the design, implementation, verification and comparison of the original algorithm and an improved version of it. The changes made in the improved version are described in Section 3. Both versions run from the same Docker image on Kubernetes as a StatefulSet. The improved version is a subclass of the original node class, and an environment variable selects which one runs.

The improved version will bring the worst-case message complexity, where a single process initiates the election, down to $O(n)$ instead of the original $O(n^2)$. The implementation details will be laid out in coming sections.

3 Methods and Materials

In order for the Bully Algorithms to work the intended way Garcia-Molina lists some assumptions that should be true. This is the case for both the original version and the improved [1, pp. 49–50]. We consider n processes $P_0, \ldots, P_{n-1}$ to have unique, totally ordered IDs and they should know the IDs of all the other processes. Processes fail by crashing, and they may later restart. Messages are never generated spontaneously, arrive uncorrupted and in the order they were sent. All those assumptions plus some more are required, and both Bully Algorithm versions in this report rely on those being true.

3.1 The original Bully Algorithm

We implement the Bully Algorithm in the form described by van Steen and Tanenbaum [6], which simplifies Garcia-Molina's original messages, and refer to it as the original version (*classic* in the figures). The original Bully Algorithm uses four message types that the processes use to communicate: *ELECTION*, *OK*, *COORDINATOR*, *HEARTBEAT*. Here *ELECTION* is sent upwards from a node to announce an election. *OK* is a reply from a higher active process that tells the announcer to withdraw, as it's no longer part of the election. The higher process then starts an election of its own. *COORDINATOR* is broadcast to all other nodes by the winner. *HEARTBEAT* is sent out from the coordinator to all nodes periodically, and that is what the other nodes use to detect coordinator failure. In this algorithm, the initiator causes every higher process to run an election on its own, and this is what makes the original algorithm expensive message-complexity wise. This means that if the lowest ID node initiates the election, we have the worst case scenario with $O(n^2)$ message complexity. This is shown in (1):

$$M_{\text{orig}}^{\text{worst}} = \underbrace{\frac{n(n-1)}{2}}_{\text{Election}} + \underbrace{\frac{(n-1)(n-2)}{2}}_{\text{Ok}} + \underbrace{(n-1)}_{\text{Coordinator}} = n(n-1) \in O(n^2). \tag{1}$$

3.2 The improved Bully Algorithm

This subsection will list the improvements made method-wise in the improved Bully Algorithm that will bring down the message complexity. It mainly contains three changes that do that:

(1) Nomination of the highest "OK" responder. The initiator learns from its "OK" responses who the highest active node is, so the cascade of elections, one of the most expensive parts of the original algorithm, can be avoided. So a process that receives *ELECTION* from a lower process answers *OK* and stops there. The initiator then collects the responders, finds the highest ID and sends one *COORDINATOR* message to the highest one. The nominee then makes the broadcast downwards to all nodes. Here the initiator stays in the *WAITING* state until the nominee's own broadcast message comes. This ensures that Assertion 1 isn't violated [1].

(2) Fallback to the next candidate. If the nominee (highest ID) crashes between answering *OK* and being nominated, the initiator removes it from its *candidate* set and nominates the next highest. If the set becomes empty it declares victory itself. The original version would start over.

(3) Priority delay before starting election. This improvement is borrowed from *The Invitation Election Algorithm* [1]. The reason for this improvement is to handle if multiple processes notice the coordinator failure at once or very close to each other. The idea is to introduce a delay based on hierarchy for when they should start their election. So on a heartbeat timeout a node therefore waits:

$$delay_i = rivals_i \cdot \Delta$$

where Δ is a fixed step and $rivals_i$ is the number of processes with a higher ID than P_i, excluding the coordinator that just failed. This means that the lower the ID, the longer it waits to initiate an election. This improvement ensures that it's mainly some of the highest IDs that start the election and in the best case the highest one. The highest active node has $rivals_i = 0$ and starts almost instantly. Its *COORDINATOR* message reaches the others before their delays expire and cancels their pending election starts.

With the improvements above, the improved Bully Algorithm has the following message complexity in the worst case, where the lowest process initiates the election alone:

$$M_{\text{imp}}^{\text{worst}} = \underbrace{(n-1)}_{\text{Election}} + \underbrace{(n-2)}_{\text{Ok}} + \underbrace{1}_{\text{nomination}} + \underbrace{(n-1)}_{\text{Coordinator}} = 3(n-1) \in O(n). \tag{2}$$

If several processes start an election at the same time, every one of them still sends ELECTION to all higher processes, so the message count is $O(n^2)$ in that case. After a coordinator failure, the priority delay makes this unlikely, but it does not apply at start-up.

4 Design and Implementation

4.1 Design

4.1.1 Architecture

To make implementation and testing seamless, each process runs in a *Docker* container on *Kubernetes*, where each pod gets an ID (the Kubernetes setup will be described in *P1: Kubernetes*). The unit tests (`testing/test_bully_node.py`) check the worst-case message counts (1) and (2)

deterministically, and a system test (`testing/test_system.py`) runs both versions with five nodes in Docker Compose and checks crash handling and the worst-case message count.

4.1.2 Communication between nodes

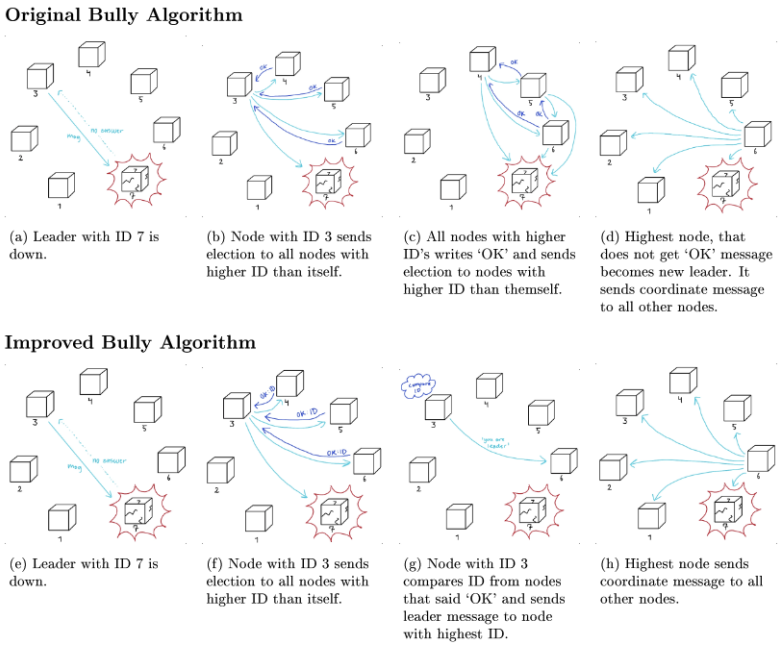


Figure 1: Communication diagrams

The two versions of the Bully Algorithm handle communication between the nodes in the system differently. Figure 1 above shows the communication for both versions. The first two steps are the same in both versions. In the example above, Node 3 sends *ELECTION* to all nodes with a higher ID when it detects a failure in the coordinator (Node 7). Nodes 4, 5 and 6 answer with *OK*. The difference in the two versions is that in the original version Nodes 4, 5 and 6 start their own elections. In the improved version, they don't. Instead Node 3 picks the highest node that answered and nominates it. Node 6 then broadcasts *COORDINATOR* to all the other nodes, which is done in both versions. This example shows a case with a low ID initiating and finishing the election, which is less likely in the improved version. But it shows that the version can still handle it.

4.1.3 Node behavior

Figure 2 shows the state of a single process. The black arrows are relevant in both versions, and the red arrows+text are only used in the improved version. We see the three different states and how a process transitions between them. A process starts an election at start-up and whenever it stops receiving heartbeats from the coordinator. In ELECTION it sends ELECTION to all processes with a higher ID. If no one answers OK within T_{ok}, it becomes coordinator itself, which is drawn as a separate box: NORMAL with its own ID as `coordinator_id`. If a higher process answers, it moves to WAITING and returns to NORMAL when a COORDINATOR message arrives. In the original version, a process also starts its own election whenever it receives

ELECTION from a lower process, which causes the cascade of elections. In the improved version, a process only answers OK, and after noticing a failure it waits a priority delay $rivals_i \cdot \Delta$ before starting an election. The initiator collects the processes that answered OK in a set and nominates the highest of them from WAITING. If the nominee does not answer, it tries the next one, and if the set is empty, it becomes coordinator itself.

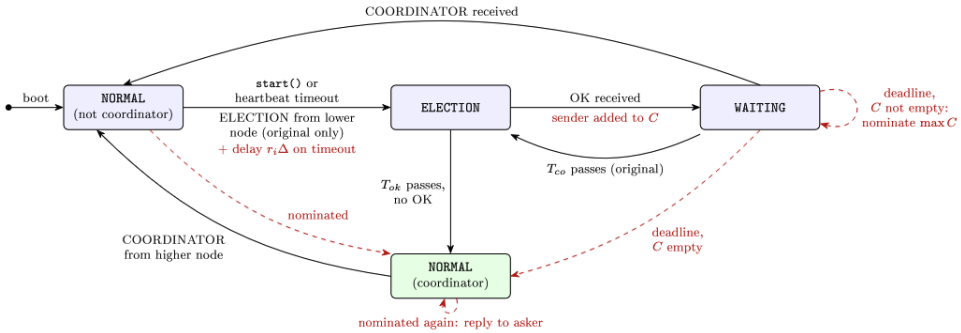


Figure 2: UML State Machine of a single process

4.2 Implementation

4.2.1 Code structure

The algorithms are written in Python as a class and a subclass. The main class is `BullyNode` which is the original version, and it gets `send` and `clock` injected which means that it can run both on a pod and in the unit tests. It then holds its public API functions, handlers, helpers, etc. How this is implemented with Docker and Kubernetes will be in 'P1: Kubernetes'.

The improved version is a subclass of `BullyNode` called `ImprovedBullyNode`, and it overrides five functions and adds three. Everything else is shared which means that the differences in the measurements come from the algorithm itself.

Both algorithms have the shared functions implemented in `BullyNode` which follows the standard algorithm methods and can be seen in the attached Source code [5]. The section below highlights some of the most relevant code from the **Improved Bully Algorithm** as it highlights the main differences of the two versions.

4.2.2 Relevant Code snippet

Listing 1 shows the most important parts of the improved version. The priority delay of election starts is computed in `_election_delay` and `tick` handles both nominations and delayed starts. In `_on_election` a node that answers *OK* also resets its heartbeat timer, which stops it from starting its own election while a lower node's election is running. This is one of the improvements in the improved version.

```
def _election_delay(self):
    # the coordinator that failed is not counted as a rival
    rivals = sum(1 for p in self.higher_peers()
                 if p != self.coordinator_id)
    return rivals * self.t.election_delay_step

```

```
def tick(self):
    now = self.clock()
    if self.state == State.WAITING and now >= self._deadline:
        if self._candidates:                # nominate the highest
            winner = self.find_winner()
            self._candidates.discard(winner)
            self._announce_coordinator(winner)
            self._deadline = now + self.t.coordinator
        else:                               # nobody left: I win
            self.declare_victory()
    elif (self.state == State.NORMAL and not self.is_coordinator
          and now - self._last_heartbeat > self.t.heartbeat_timeout):
        if self._election_at is None:       # start the delay
            self._election_at = now + self._election_delay()
        if now >= self._election_at:
            self.start_election()
    else:
        self._election_at = None            # cancel the delay
        super().tick()

def _on_election(self, msg):
    if msg.sender_id < self.id:
        self._send(msg.sender_id, MsgType.OK)
        self._last_heartbeat = self.clock()   # no election of my own
```

Listing 1: Key parts of `ImprovedBullyNode` (shortened).

5 Experiments

This section compares the original bully election algorithm with our improved version through concrete experiments. The goal is to find the difference between the two algorithms on three points: is the correct leader selected, how many messages are sent per election, and how much time does each election take. The experiments also check whether the measured message counts match the theoretical worst cases from Section 3 for both the original and the improved version.

5.1 Experiment setup

The experiments run on the deployment explained in section 4.1.1, with a Kubernetes cluster created using kind, consisting of a control-plane node and two worker nodes. Each process (P_i) is a pod named `bully-`i running from the same Docker image. The algorithm used is chosen using an environment variable, which ensures that both algorithms run on identical infrastructure, with all the same timeouts and node classes. See Table 1 for the parameters.

All experiments are run from the host by the script docker/experiment.py. This script controls the cluster using kubectl. Before every run it resets the message counter for all processes, then stages the scenario and waits until all live bully nodes agree on a leader. Then it collects the state and counters from every process.

In order to test crashes on any process, and not only the one with the highest ID, every process exposes the test endpoints /crash and /recover. After /crash, the process stops running

Table 1: Parameters used in all experiments.

Parameter	**Value**
T_{ok} (wait for OK after ELECTION)	3 s
Wait for COORDINATOR after OK	5 s
Heartbeat interval / heartbeat timeout	2 s / 5 s
Priority delay step Δ (improved version only)	4 s
Number of processes n	5, 10, 15, 20
Repetitions per scenario, version and n	3 (144 runs in total)

its algorithm, rejects all incoming messages and sends nothing, so to the other processes it is indistinguishable from a real crash. After /recover, the process restarts with an empty state and starts an election. In the failover and worst-case scenarios, the coordinator is not crashed with /crash. Instead, its pod is removed by scaling the StatefulSet down to $n - 1$ replicas.

5.2 Scenarios

Every scenario starts from a stable system where the coordinator is P_{n-1}, except the start-up scenario where there is no coordinator. See Table 2 for a description of each scenario.

Table 2: Experiment scenarios.

Scenario	**What happens**	**Expected leader**
Start-up	All n processes start at the same time	P_{n-1}
Failover	The coordinator crashes, and all processes detect it through the heartbeat timeout	P_{n-2}
Worst case	The coordinator crashes, and only the lowest process P_0 starts the election	P_{n-2}
Non-coordinator crash	A random process that is not the coordinator crashes	P_{n-1} (no election)
Two highest crash	P_{n-1} and P_{n-2} crash at the same time	P_{n-3}
Coordinator recovery	The coordinator crashes, a new one is elected, and the old coordinator recovers	P_{n-1}

5.3 Metrics

In order to measure and compare the different algorithms, we must have some set metrics, so our comparison is based on three main points.

Election outcome: After every run, all live processes must agree on the same coordinator, and this must be the live process with the highest ID.

Message count: We take the sum of all messages (ELECTION, OK, COORDINATOR) sent by processes during the run. This excludes heartbeats as they are used for failure detection and are sent at a constant interval.

Election time: Defined as the time from when the first process starts the election, until the last process knows the new coordinator. Again the failure detection time is not included as this is set by the heartbeat as well and not affected by the algorithm.

6 Results

6.1 Election outcome

In all 144 runs, all live processes agreed on the same, correct coordinator, so both algorithms work in all scenarios tested.

6.2 Message complexity

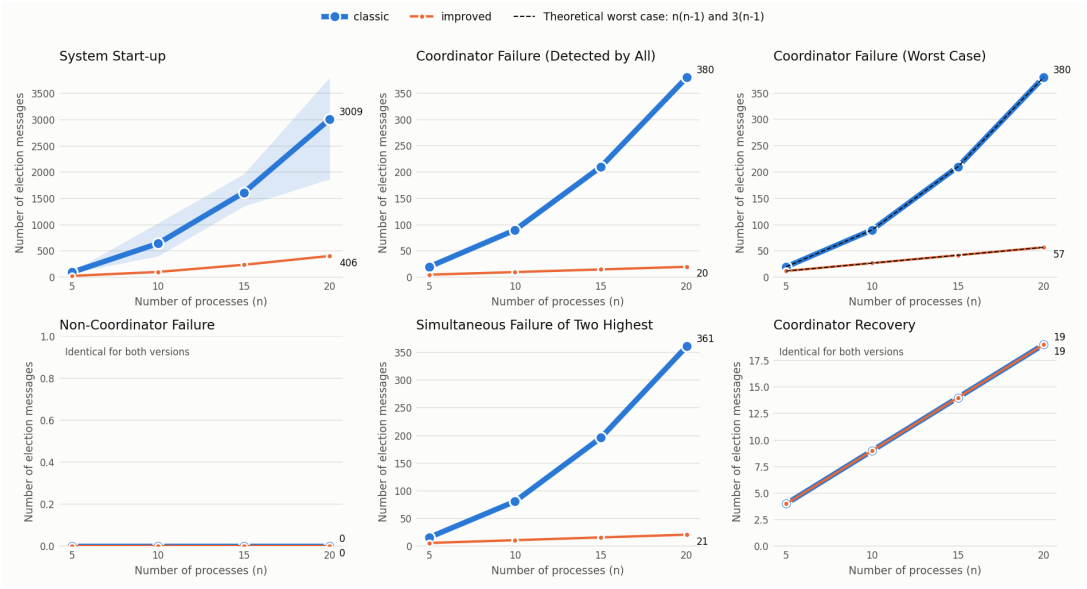


Figure 3: Election messages as a function of n

Note: Mean of 3 runs; the shaded area shows min–max. Classic is the thick blue line, improved the thin orange line. The dashed lines show the theoretical worst-case values $n(n-1)$ and $3(n-1)$.

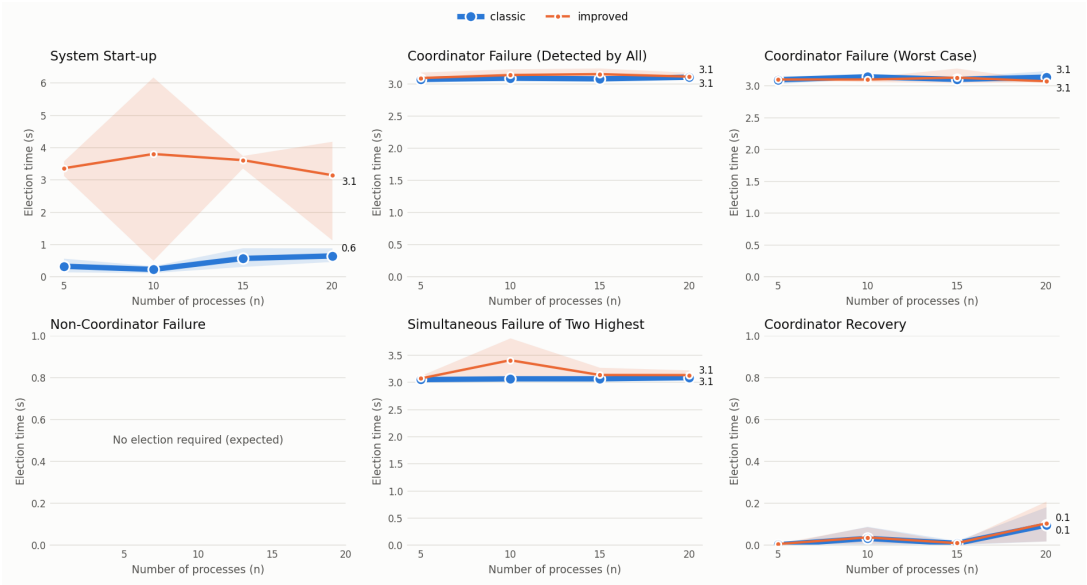


Figure 4: Election time as a function of n

Note: Election time per scenario as a function of n (mean of 3 runs, min–max shaded).

7 Discussion

The results show that the measured results correspond to the expected results for both the original and the improved Bully Algorithm.

7.1 Time- and message complexity

When starting the system, the number of election messages grows quadratically with the number of nodes for both versions, but much faster for the original version than for the improved version. This is because of the nomination, since a process that receives ELECTION only answers OK and does not start its own election, so the cascade of elections is avoided. The priority delay plays no role here, since it is only used after a heartbeat timeout and not at start-up. Conversely, the election time increases. This is because an initiator in the improved version waits the full T_{ok} before it nominates the highest responder, whereas in the original version the highest process starts its own election and declares victory immediately. The shaded area shows the spread between the fastest and slowest of the three runs. At start-up, this spread is large for the improved version, because the election time depends on the order in which the pods come up. If a process starts after the highest process has broadcast COORDINATOR, it starts its own election and waits T_{ok} before it nominates.

This only seems to be a problem at start-up, as we do not see any significant time increase in the failover, worst-case and coordinator recovery scenarios. When the two highest processes crash, we see a slight increase in time as well, but nothing significant for the system.

The message complexity of the improved algorithm has improved or is similar to the original algorithm, showing that our optimization of the message complexity is a success. We see a significant improvement at start-up and in the failover, worst-case and two-highest-crash scenarios.

7.2 Limitations

Several limitations should be considered when interpreting our results. The number of nodes tested was limited to 5-20 nodes, and each scenario was repeated 3 times. Larger systems might show different scaling behavior, and more repetitions would give more reliable estimates. We assumed reliable communication channels, and did not count heartbeats. The $O(n)$ bound only holds when a single process initiates the election. At start-up, all processes start an election at the same time, and the improved version is still $O(n^2)$.

Both time and message complexity are important factors, when optimizing an algorithm. As the results show, we have focused on improving the message complexity. We do reflect on the time complexity, but it is not a priority for this optimization.

8 Conclusion

We implemented both the original Bully Algorithm and an improved version in Python. The improved version introduces three changes: nomination of the highest *OK* responder, fallback to the next candidate if the nominee crashes, and a priority delay ($rivals_i \cdot \Delta$) before starting an election. Both algorithms elected the correct coordinator in all 144 runs across all six scenarios,

so the improvements do not compromise correctness. The worst-case message complexity with a single initiator was reduced from $O(n^2)$ to $O(n)$, and the experimental results match these theoretical bounds. The reduction in messages comes at a cost in election time at start-up, caused by the initiator waiting T_{ok} before nominating the highest responder. The improved algorithm is therefore best suited for systems where network traffic is costly and the system runs for long periods without restarts.

9 Perspectives

Our results agree with earlier work showing that the message complexity of the Bully Algorithm can be reduced from $O(n^2)$ to $O(n)$. Our nomination mechanism is close to the *GRANT* approach of Kordafshari [2], but we extend it with a fallback to the next candidate. To handle simultaneous elections, we used a priority delay inspired by Garcia-Molina's Invitation Algorithm [1], while Murshed and Allen [3] use a tie breaker time for the same purpose. Modern systems such as Raft [4] instead use randomized election timeouts. These give a shorter wait on average than our deterministic delay, which grows linearly with the number of higher processes.

References

[1] Hector Garcia-Molina. "Elections in a Distributed Computing System". In: *IEEE Transactions on Computers* C-31.1 (1982), pp. 48–59.

[2] M. S. Kordafshari et al. "Modified Bully Election Algorithm in Distributed Systems". In: *Proceedings of the WSEAS Conferences*. 2005.

[3] Md. Golam Murshed and Alastair R. Allen. "Enhanced Bully Algorithm for Leader Node Election in Synchronous Distributed Systems". In: *Computers* 1.1 (2012).

[4] Diego Ongaro and John Ousterhout. "In Search of an Understandable Consensus Algorithm". In: *Proceedings of the 2014 USENIX Annual Technical Conference*. 2014.

[5] SANS, Team 8. *Bully Algorithm: Source Code*. 2026. URL: `https://github.com/Studiegruppe-SANS/distributed_systems_P1_leader_election`.

[6] Maarten van Steen and Andrew S. Tanenbaum. *Distributed Systems*. 4th ed. Version 4.01. Maarten van Steen, 2023.